# CUDA for Triton Programmers

A Comprehensive Course — Every Topic with a TL;DR First

June 2026

**Abstract**

A complete first pass through CUDA, written for someone who already writes Triton. The premise throughout: **Triton hands you a tile; CUDA makes you build that tile out of threads.** Every section opens with a one-or-two-sentence **TL;DR** (read just those for a fast skim), followed by the full explanation, code, and the mapping back to the Triton concept you already know. Read top to bottom for the course, or jump via the contents and skim the TL;DRs.

## Contents

# 1 How GPUs execute work

**TL;DR.** A GPU is a throughput machine: thousands of tiny threads run the same code on different data. You already know this from Triton — the new part is that CUDA lets you see and control the individual threads inside a tile.

A CPU spends transistors making *one* thread fast (big caches, branch prediction, out-of-order execution). A GPU spends them on *many* simple lanes that run in parallel, and hides memory latency not with caches but by having a huge pool of threads ready to run — when one stalls on memory, another runs. This is the "Single Instruction, Multiple Threads" (SIMT) model.

Your program has two sides. The **host** (CPU + its RAM) orchestrates; the **device** (GPU + its VRAM) computes. The host allocates device memory, copies data in, launches **kernels** (GPU functions), and copies results back. In Triton you felt only a sliver of this (the `@triton.jit` function and its launch grid); CUDA makes the whole host/device dance explicit.

# 2 The core mental shift: program-level vs. thread-level

**TL;DR.** One Triton *program* = one CUDA *thread block*. In Triton you wrote code for a whole tile; in CUDA you write code for one thread, and the tile behavior emerges from many threads cooperating.

In Triton you reason as one program instance: `pid = tl.program_id(0)` picks your tile, and you operate on whole vectors (`tl.load`, `tl.dot`, `tl.sum`). The compiler turns each program into a block of `num_warps`×`32` threads and synthesizes all the per-thread detail.

CUDA removes that automation. A kernel launches as a **grid** of **blocks** of **threads**; you write what a single thread does. The tile-level operations you got for free are now things you assemble explicitly from threads, warps, and shared memory. Almost everything else in this course is a consequence of this one change.

## 3 Threads, blocks, grids, and indexing

**TL;DR.** Threads are grouped into blocks, blocks into a grid. Each thread computes a global index to find its data — this replaces Triton's `pid * BLOCK + tl.arange(0, BLOCK)`.

The hierarchy:

- **Thread** — one lane of execution; the unit Triton hid.
- **Block** — a group of threads (up to 1024) that share fast on-chip memory and can synchronize. *Equivalent to one Triton program.*
- **Grid** — all blocks launched by one kernel. *Equivalent to your Triton launch grid.*

Blocks and threads can be 1D/2D/3D (handy for vectors/images/volumes). Every thread reads its coordinates from built-ins: `threadIdx` (within block), `blockIdx` (within grid), `blockDim` (threads per block), `gridDim` (blocks per grid). The canonical 1D global index:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

This is literally Triton's `pid*BLOCK_SIZE + tl.arange(0,BLOCK_SIZE)` — except Triton produced a *vector* of indices (the whole tile) while each CUDA thread produces *one* of those indices.

## 4 The memory hierarchy

**TL;DR.** Registers (per-thread, fastest) → shared memory (per-block, fast, on-chip) → global memory (huge, slow VRAM). Triton chose where things lived; in CUDA you place them. Speed = keeping reused data out of global memory.

From fastest/smallest to slowest/largest:

- **Registers** — private per thread; hold local variables and accumulators. Limited in number; using too many lowers occupancy (§16).
- **Shared memory** (`__shared__`) — per-block scratchpad, on-chip, ∼100× faster than global. Threads in a block use it to cooperate and cache reused data. This is the scratch Triton allocated for you in `tl.dot`/reductions.
- **L1/L2 caches** — automatic; L2 is shared across the GPU.
- **Global memory** — the multi-GB VRAM; visible to all threads, high latency. Your inputs/outputs live here.
- **Constant & texture memory** — small, read-only, cached; good for values every thread reads.
- **Local memory** — per-thread spillover that physically lives in slow global memory (e.g. register spills or indexed local arrays). Avoid.

The entire performance game is moving data up this pyramid and reusing it before it falls back down — exactly what tiling does.

## 5 Writing and launching a kernel; compilation

**TL;DR.** Write a `__global__` function, launch it with `kernel<<<blocks, threads>>>(args)`, compile with `nvcc`. The five steps are allocate, copy-in, launch, copy-out, free.

```cpp
__global__ void add_kernel(const float* x, const float* y, float* out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;  // one element per thread
    if (i < n) out[i] = x[i] + y[i];                // the "mask"
}

void launch(const float* hx, const float* hy, float* hout, int n) {
    float *dx, *dy, *dout;  size_t bytes = n * sizeof(float);
    cudaMalloc(&dx, bytes); cudaMalloc(&dy, bytes); cudaMalloc(&dout, bytes); // 1
    cudaMemcpy(dx, hx, bytes, cudaMemcpyHostToDevice);                        // 2
    cudaMemcpy(dy, hy, bytes, cudaMemcpyHostToDevice);
    int threads = 256, blocks = (n + threads - 1) / threads;
    add_kernel<<<blocks, threads>>>(dx, dy, dout, n);                          // 3
    cudaMemcpy(hout, dout, bytes, cudaMemcpyDeviceToHost);                    // 4
    cudaFree(dx); cudaFree(dy); cudaFree(dout);                               // 5
}
```

`__global__` marks a kernel (host-launchable, device-run); `__device__` marks a helper callable only from the device; `__host__` is ordinary CPU code. Compile with `nvcc file.cu -o prog`: it separates device code (compiled to PTX/SASS for the GPU) from host code (handed to your C++ compiler). Launches are **asynchronous** — the host continues immediately; the next `cudaMemcpy` or `cudaDeviceSynchronize()` blocks until the GPU finishes. (In Triton, the launch was async too; you just rarely had to think about it.)

**Triton vs CUDA, same kernel**

```python
@triton.jit
def add_kernel(x_ptr, y_ptr, o_ptr, n, BLOCK: tl.constexpr):
    offs = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)  # a tile of indices
    mask = offs < n
    tl.store(o_ptr + offs, tl.load(x_ptr+offs, mask=mask) + tl.load(y_ptr+offs, mask
        =mask), mask=mask)
```

The Triton tile (`tl.arange`) becomes one index per CUDA thread; the `mask` becomes an `if`.

## 6 Warps and SIMT execution

**TL;DR.** Threads run in lock-step groups of 32 called warps. If threads in a warp branch differently (*divergence*), both paths run serially. Triton hid warps; now branch shape matters.

A block's threads run in **warps of 32 threads** that move in lock-step: at each step all 32 lanes execute the *same* instruction, just on their own data. Picture 32 dancers sharing one set of called-out moves.

**Warp divergence — paying for both branches.** The trouble starts at an `if/else`. Since the 32 lanes share one instruction stream, they cannot run two different instructions at once. If some lanes take the `if` and others the `else`, the hardware runs the two branches *one after the other*: first the `if`-lanes work while the `else`-lanes sit idle (masked off), then they swap. You pay for *both* paths in sequence — the warp is as slow as the sum of the branches its lanes take. The key simplifications:

- **It only matters *inside* a warp.** If all 32 lanes agree, there is zero penalty. If two *different* warps take different paths, that is completely free — warps are scheduled independently.

- **So branch on something uniform across the warp.** `if (blockIdx.x == 0)` — every lane shares `blockIdx`, so the whole warp agrees: no divergence. `if (threadIdx.x % 2)` — lanes disagree within the warp: $\sim 2\times$ cost.
- **Loops diverge too.** A data-dependent loop runs until the *last* lane finishes, so an uneven trip count makes all 32 lanes wait for the slowest.
- Triton hid this by compiling your masks to predication, so you never reasoned about which lanes were active. In CUDA the branch shape is yours to manage.

**The warp is the real scheduling unit.** Occupancy, latency hiding, and memory coalescing are all defined per warp — so block sizes should be a multiple of 32, and "neighboring threads" really means "the 32 lanes of one warp."

**Why it physically serializes.** A warp shares *one* instruction-issue unit — one fetch/decode/scheduler (classically one program counter) driving 32 lanes, because GPUs amortize expensive control hardware across cheap datapath. Only one instruction can be issued to the warp per cycle, so when lanes need *different* instructions (the two sides of a branch) the hardware cannot deliver both at once: it runs the `if` path with the non-taken lanes masked off (their ALUs idle), then the `else` path with the complementary mask — time = sum of both paths. The masks recombine at reconvergence (the post-dominator), which bounds the cost. This is the same bargain as a bank conflict: a cheap *shared* resource (here one issue slot; there one SRAM port) runs 32-wide when lanes are uniform and serializes when they collide. (Volta's per-thread PCs let divergent lanes interleave, but there is still one issue slot per cycle — so divergence is more flexible, not free.)

## 7 Warp-level primitives

**TL;DR.** Threads in a warp can swap registers directly with shuffle instructions — no shared memory needed. This is how Triton's `tl.sum`/`tl.max` reduce inside a warp.

**Warp shuffles** (`__shfl_down_sync`, `__shfl_xor_sync`) let the 32 lanes exchange register values in one instruction, enabling register-only reductions and broadcasts. **Vote/ballot** (`__ballot_sync`, `__any_sync`) gather per-lane predicates into a bitmask — useful for stream compaction and divergence-aware logic. **Cooperative groups** give a tidier API over the same hardware. A warp-level sum:

```
__device__ float warpReduceSum(float v) {
    for (int off = 16; off > 0; off >>= 1)
        v += __shfl_down_sync(0xffffffff, v, off);  // tree reduce within 32 lanes
    return v;  // lane 0 holds the warp total
}
```

## 8 Shared memory and bank conflicts

**TL;DR.** Shared memory is a fast per-block scratchpad you manage by hand: load global → shared, `__syncthreads()`, then compute. Watch bank conflicts (pad arrays). Triton did all of this silently for `tl.dot`.

Declare it (`__shared__ float tile[TS][TS];`), cooperatively load global into it (each thread brings a piece), call `__syncthreads()` so everyone's writes are visible, then read. The one performance trap is **bank conflicts**.

**What a bank is.** Shared memory is split into **32 banks** — picture 32 checkout lanes. A warp's 32 threads can grab 32 values in a *single* step, but only if each thread's value sits in a *different* bank. Banks are assigned by address: consecutive 4-byte words go to consecutive banks and wrap every 32 words, so the word at index $i$ lives in bank $i$ mod 32.

**The conflict.** If two threads in the warp want different words that fall in the *same* bank, the hardware serves them one after another — a **2-way conflict** (2× slower); $k$ threads on one bank is a $k$-way conflict ($k$× slower). (One free exception: if all threads read the *same* address, that is a **broadcast**, not a conflict.)

**The classic example.** Take `__shared__ float tile[32][32]` and read down a *column*: thread $t$ reads `tile[t][c]`. Each row is 32 words wide, so `tile[0][c], tile[1][c], …` are all exactly 32 words apart — i.e. all in the *same* bank. That is a 32-way conflict; the warp's reads fully serialize. Reading across a *row* (`tile[r][t]`) instead hits 32 different banks — no conflict.

**The fix: pad by one.** Declare `tile[32][33]` (i.e. `[TS][TS+1]`). Now each row is 33 words wide, so stepping down a column advances the bank by 33 mod 32 = 1 every time — the 32 column elements spread across all 32 banks and the conflict disappears:

```
[32][32]: column c -> banks  c, c, c, ..., c      (same bank -> 32-way conflict)
[32][33]: column c -> banks  c, c+1, c+2, ...     (all different -> no conflict)
```

You waste one column of shared memory and get full-speed column access in return. Triton applied exactly this kind of swizzle for you automatically.

**Why it physically serializes.** A bank is a single-ported SRAM array — one address decoder and one set of bit-lines/sense-amps. Reading a word asserts exactly *one* word-line; you cannot drive two different rows onto the shared bit-lines at once without corrupting the read, so a bank delivers one word per cycle. Two distinct addresses in one bank must therefore be decoded and sensed back-to-back ($k$ distinct addresses = $k$ cycles); the *same* address is a single read fanned out to all lanes (broadcast — free). 32 single-ported banks give a full warp's bandwidth ($32 \times 4\,\mathrm{B} = 128\,\mathrm{B}$/cycle) in the conflict-free case, whereas true multi-porting would remove conflicts but cost far too much area/power. Same trade as warp divergence: a cheap shared resource (one port) is fast when lanes spread out and serial when they collide.

## 9 Synchronization and atomics

**TL;DR.** `__syncthreads()` is a per-block barrier (wait for everyone before reading shared data). Atomics (`atomicAdd`) let many threads safely update one address. Maps to Triton's implicit barriers and `tl.atomic_add`.

`__syncthreads()` synchronizes all threads *in a block* — mandatory between writing shared memory and reading what neighbors wrote. There is no cheap global barrier across the whole grid within a kernel; separate phases usually mean separate kernel launches (or cooperative-groups grid sync, with caveats). **Atomics** (`atomicAdd`, `atomicCAS`, …) serialize concurrent updates to a location for histograms, counters, and reductions into global memory; correct but contended, so prefer block-local reductions first, then one atomic per block.

**`__syncthreads()` vs. `cudaDeviceSynchronize()`.** Both say "sync," but the scopes are completely different. `__syncthreads()` is *device-side*, called *inside* a kernel: a barrier across the threads of *one block* — each thread waits until all threads in its block reach the line (used between shared-memory writes and reads). It does *not* synchronize across blocks. `cudaDeviceSynchronize()` is *host-side*, called from CPU code: it blocks the *CPU* until *all* previously launched GPU work (every kernel and copy on the device) has finished. So one coordinates threads *within a block, mid-kernel*; the other makes the *host wait for the whole GPU*. You cannot call `__syncthreads()` from the host, you essentially never call `cudaDeviceSynchronize()` from a kernel, and the latter is a heavy global barrier to keep out of hot loops (prefer stream/event sync).

## 10 Memory coalescing and access patterns

**TL;DR.** Make consecutive threads read consecutive addresses — the hardware then fuses a warp's 32 loads into a few wide transactions. Strided/transposed access wastes most of your bandwidth. Triton chose coalesced patterns for you.

Global memory is delivered in wide bursts (e.g. 32- or 128-byte segments). When the 32 lanes of a warp touch a contiguous, aligned span, the hardware **coalesces** them into the minimum number of transactions; scattered access issues many. The rule of thumb: thread $i \leftrightarrow$ element $i$. Reading a matrix column-major across threads (stride = row length) is the classic bandwidth killer — stage through shared memory to fix it. **Vectorized loads** (`float4`) move 16 bytes per thread and cut transaction count further. All of this was automatic in Triton's `tl.load`.

## 11 Worked example: parallel reduction (`tl.sum` by hand)

**TL;DR.** Summing an array fast = each block reduces its chunk in shared memory (tree), then warp-shuffles the last 32, then one atomic to global. This is what `tl.sum` compiles to.

```
__global__ void reduceSum(const float* in, float* out, int n) {
    __shared__ float s[256];
    int t = threadIdx.x, i = blockIdx.x * blockDim.x + t;
    s[t] = (i < n) ? in[i] : 0.0f;
    __syncthreads();
    for (int stride = blockDim.x/2; stride > 32; stride >>= 1) { // tree in shared
        if (t < stride) s[t] += s[t + stride];
        __syncthreads();
    }
    float v = s[t];
    if (t < 32) v = warpReduceSum(v + s[t+32]); // finish in registers (no sync)
    if (t == 0) atomicAdd(out, v);              // one atomic per block
}
```

Three ideas stack here: shared-memory tree reduction, warp-shuffle finish (no barrier needed inside a warp), and a single atomic per block to combine partials. Triton expressed the whole thing as `tl.sum(x, axis=0)`.

## 12 Reduction strategy: split-reduction vs. `atomicAdd`

**TL;DR.** To reduce a big axis, do *not* fire one atomic per element. Reduce privately first (registers → warp/shared), then finish with either one atomic *per block* or a small second kernel. Per-element atomics cost you contention, nondeterminism, and accuracy. This is exactly Triton's split-reduction vs. `tl.atomic_add` choice.

The reduction in §11 hid a decision that recurs whenever a kernel accumulates many contributions into far fewer outputs: how does the *cross-worker* combine happen? Two patterns:

- **Split (staged) reduction.** Each worker reduces its slice privately (register accumulator, then a warp/shared reduction), writes a *partial*, and a second pass — or a single atomic per block — combines the partials.
- **Atomic accumulation.** Each worker `atomicAdd`s its contribution straight into the shared output.

**A motivating shape.** Suppose two sequences are stored $(B, L, D)$ and you want `out`$[b] = \sum_i \sum_j f(A_{b,i}, B_{b,j})$ *without* ever materializing the $(B, L, L)$ pairwise object. You stream the inner $L$ into a register accumulator (fine under either pattern); the real question is how to reduce the *outer* $L$. Either write a linear $(B, L)$ partial and reduce it (split), or `atomicAdd` each $i$ into `out`[b] (atomic).

**Why split usually wins.**

- **Contention.** Atomics into `out`[b] from all $L$ outer-workers serialize on one address — and the *more* you parallelize, the worse it gets. Split has none.
- **Determinism & numerics.** Float `atomicAdd` accumulates in nondeterministic order ⇒ non-reproducible runs and $O(L)$ error growth; a tree/pairwise reduction is reproducible with $O(\log L)$ error. For training, reproducibility alone often decides it.
- **The partial is cheap.** It is linear — typically smaller than the inputs themselves — so the quadratic-free goal is untouched.

```
// ANTI-PATTERN: one atomic per inner element -> L-way contention on out[b]
for (int j = 0; j < L; ++j) atomicAdd(&out[b], f(A[b][i], B[b][j]));

// BETTER: accumulate privately, then ONE atomic per block (cf. reduceSum above)
float acc = 0.f;
for (int j = 0; j < L; ++j) acc += f(A[b][i], B[b][j]); // register accumulator
acc = blockReduceSum(acc);                              // warp+shared reduction
if (threadIdx.x == 0) atomicAdd(&out[b], acc);          // one atomic per block
```

**When atomics are fine:** low fan-in (few contributions per output), or you have *already* reduced to one atomic per block so contention is $O(\#\text{blocks})$, not $O(\#\text{elements})$. The cardinal rule holds either way: never atomic-per-inner-element — reduce hierarchically first, then at most one atomic per block. In Triton, `tl.atomic_add` is fine *once per program instance*; accumulating per inner element is the anti-pattern, and the staged alternative is the same split-K idea Triton uses to spread a reduction across programs and combine the partials.

## 13 Worked example: tiled matrix multiply (`tl.dot` by hand)

**TL;DR.** Block owns an output tile; loop over K loading A/B sub-tiles into shared memory, multiply-accumulate into registers, write out. That K-loop + `tl.dot` you wrote in Triton *is* this — now explicit.

```
#define TS 16
__global__ void matmul(const float* A, const float* B, float* C, int N) {
    __shared__ float As[TS][TS], Bs[TS][TS];
    int r = blockIdx.y*TS + threadIdx.y, c = blockIdx.x*TS + threadIdx.x;
    float acc = 0.0f;                                  // register accumulator
    for (int k0 = 0; k0 < N; k0 += TS) {               // the Triton K-loop
        As[threadIdx.y][threadIdx.x] = A[r*N + (k0+threadIdx.x)]; // stage tiles
        Bs[threadIdx.y][threadIdx.x] = B[(k0+threadIdx.y)*N + c];
        __syncthreads();
        for (int k = 0; k < TS; ++k)                   // the "tl.dot"
            acc += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        __syncthreads();
    }
    C[r*N + c] = acc;
}
```

The accumulator lives in registers; each A/B element loaded from global is reused `TS` times from shared memory — that reuse is why tiling is fast. Real kernels add register blocking (each thread computes several outputs), vectorized loads, and Tensor Cores (§15). For production, CUTLASS/cuBLAS already do all of this.

## 14 Asynchronous copies and software pipelining (`num_stages`)

**TL;DR.** Overlap loading the next tiles with computing the current ones, using `cp.async` + double-buffered shared memory. This is exactly what Triton's `num_stages` autotuned for you.

A naive tiled loop alternates "load, sync, compute, sync," so compute waits on memory. **Software pipelining** breaks the dependency: prefetch tile $k{+}1$ into a second shared-memory buffer *while* computing on tile $k$, then rotate buffers. The enabling instruction is **`cp.async`** (Ampere+), which copies global → shared without going through registers and without blocking the thread. You issue the next-stage copies, compute the current stage, then `cp.async.wait_group` and swap. Triton's `num_stages=N` built an $N$-deep version of this pipeline automatically; in CUDA you write the buffer rotation yourself (or use CUTLASS's pipeline abstractions / `libcudacxx cuda::pipeline`).

## 15 Tensor Cores (`tl.dot`'s hardware)

**TL;DR.** Tensor Cores are dedicated units that multiply small matrix tiles in one instruction (fp16/bf16/tf32/fp8 in, fp32 accumulate). `tl.dot` lowers to these; in CUDA reach them via `wmma`/`mma` PTX or, better, CUTLASS.

Modern NVIDIA GPUs include **Tensor Cores** that compute a matrix multiply- accumulate $D = A \cdot B + C$ on small fixed tiles (e.g. $16 \times 16 \times 16$) in a single hardware op, at many times the FLOP rate of the regular FP32 pipeline. Inputs are reduced precision (FP16/BF16/TF32/FP8) with FP32 accumulation. Access them via:

- The `wmma` C++ API (`nvcuda::wmma::fragment` + load/mma/store) — approachable but limited.

- Inline `mma`/`wgmma` PTX — maximal control, used by libraries.
- **CUTLASS** — templated kernels exposing the same tile/stage knobs you know from Triton; the practical choice for custom Tensor-Core kernels.

`tl.dot` hides all of this; you only drop down when you need a fusion or schedule Triton/cuBLAS won't give you.

## 16 Occupancy and launch configuration (autotune by hand)

**TL;DR.** Occupancy = how many warps are resident per SM to hide latency. It's capped by registers/thread and shared-memory/block. Picking block size + tile size is the tradeoff Triton's autotuner searched.

Each Streaming Multiprocessor (SM) has a fixed budget of registers and shared memory. A kernel using more registers per thread or more shared memory per block fits *fewer* concurrent warps — lower **occupancy**. Enough resident warps lets the SM hide memory latency by switching to a ready warp when one stalls; too few and the SM idles. But maximizing occupancy isn't always optimal — bigger tiles (more registers/shared) can win via reuse despite lower occupancy. The knobs: **block size** (your `num_warps`×32), **tile sizes**, and register/shared usage. The CUDA Occupancy API and Nsight Compute report the limits; in Triton, `@triton.autotune` swept this surface for you.

## 17 Streams, events, and concurrency

**TL;DR.** Streams are queues of GPU work; operations in different streams can overlap. Use them to run copy and compute at the same time. Triton rarely exposed this.

Streams are a **host-side** construct: you organize *launches* with them in CPU code; a kernel never sees a stream. By default all work goes into one stream and runs in order. Create multiple `cudaStream_t` and independent work in different streams can run concurrently — classically, copy chunk $k{+}1$ (`cudaMemcpyAsync`) while *computing* on chunk $k$ and *copying out* chunk $k{-}1$. **Events** (`cudaEvent_t`) timestamp work and create cross-stream dependencies, and are the right tool for GPU timing. Pinned (page-locked) host memory (§18) plus `cudaMemcpyAsync` are required for true overlap.

**Example: a copy/compute pipeline over chunks**

```
const int S = 3;                        // ring of streams (multi-buffering)
cudaStream_t st[S];
for (int i = 0; i < S; ++i) cudaStreamCreate(&st[i]);

int chunk = n / nChunks;                // split the work into chunks
for (int c = 0; c < nChunks; ++c) {
    int s = c % S, off = c * chunk;     // round-robin across streams
    cudaMemcpyAsync(dx+off, hx+off, chunk*sizeof(float),
                    cudaMemcpyHostToDevice, st[s]);          // copy IN
    process<<<blocks, threads, 0, st[s]>>>(dx+off, dy+off, chunk); // COMPUTE
    cudaMemcpyAsync(hy+off, dy+off, chunk*sizeof(float),
                    cudaMemcpyDeviceToHost, st[s]);          // copy OUT
}
for (int i = 0; i < S; ++i) cudaStreamSynchronize(st[i]);    // host waits
// note: the 4th launch param <<<grid,block,sharedMem,stream>>> picks the stream.
```

**What it buys you.** A GPU has *separate* hardware units — the SMs (compute) and one or two DMA *copy engines* (H2D and D2H) — that can all run at once. With one stream they idle in turn; streams expose that parallelism:

```
one stream:  [---copy in---][---compute---][--copy out--]   total = sum
N streams :  [in0][in1][in2]
                  [cmp0][cmp1][cmp2]
                        [out0][out1][out2]                  total ~= max
```

Without streams the time is $T_{\text{in}} + T_{\text{compute}} + T_{\text{out}}$, and nothing starts computing until the *whole* array has copied. With a chunked, multi-stream pipeline the three engines work simultaneously, so the time collapses to roughly $\max(T_{\text{in}}, T_{\text{compute}}, T_{\text{out}})$ plus a small fill/drain — up to a 2–3× speedup when the stages are balanced, and it hides transfer latency entirely when compute dominates. A second payoff: several *small* kernels that don't individually fill the GPU can run concurrently in different streams, raising utilization. (In Triton/PyTorch this layer is managed for you — kernels enqueue on the current `torch.cuda` stream — which is why you rarely touch it; `torch.cuda.stream(...)` is the same idea wrapped.)

## 17.1 PyTorch integration: launching on the current stream

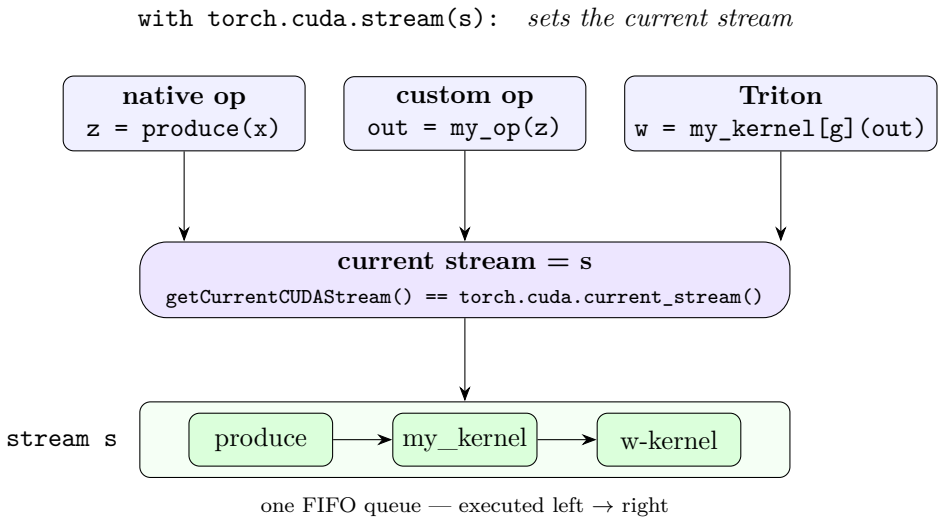
**TL;DR.** In a custom CUDA op, launch your kernel on PyTorch's *current* stream via `at::cuda::getCurrentCUDAStream()` — not the default stream — so it is ordered correctly with the surrounding ops. Triton does this for you.

PyTorch keeps a **current stream** per device, and every async operation it issues enqueues there. A hand-written kernel wrapped as a custom op must join that *same* timeline; otherwise it sits on a separate stream with no ordering against the tensors it reads and writes — a silent race. You wire this up by querying the current stream and passing it as the launch's 4th parameter:

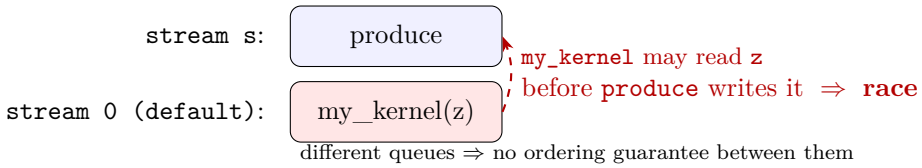
```cpp
#include <ATen/cuda/CUDAContext.h>
#include <c10/cuda/CUDAStream.h>

void my_op(torch::Tensor x, torch::Tensor y) {        // host wrapper, exposed via pybind
    int n = x.numel(), t = 256, b = (n + t - 1) / t;
    cudaStream_t s = at::cuda::getCurrentCUDAStream(); // PyTorch's current stream
    my_kernel<<<b, t, 0, s>>>(x.data_ptr<float>(), y.data_ptr<float>(), n);
    // no cudaDeviceSynchronize(): keep it async; ordering comes from the stream
}
```

**How a launch routes to the stream.** Every enqueue — a native op, your custom op, or a Triton kernel — reads the same "current stream" and lands in its one ordered queue:




If `my_op` instead launched on the *default* stream, its kernel would sit in a *different* queue with no ordering against `produce` — so it could read `z` before `produce` finished writing it:




On the Python side, `with torch.cuda.stream(s):` sets the current stream, and everything inside — native ops, this custom op (because it reads `getCurrentCUDAStream()`), and Triton kernels — lands on `s`, so they stay correctly ordered. Two rules and two gotchas:

- **Never launch on the default stream** from inside an op, and **do not synchronize** — correctness comes from being on the right stream, and a `cudaDeviceSynchronize` would serialize the whole pipeline.
- **Cross-stream tensors:** if you use a tensor on a different stream than it was allocated on, call `tensor.record_stream(stream)` so PyTorch's stream-aware caching allocator does not recycle its memory too early.
- **Internal streams:** if your op spins up its own streams for overlap, rejoin them to the current stream with an event (`cudaStreamWaitEvent`) before returning, so downstream ops wait; the `c10::cuda::CUDAStreamGuard` helper manages the current-stream swap cleanly.

This is exactly what Triton does under the hood — it grabs `torch.cuda.current_stream()` and launches on it — which is why streams never came up while you were writing Triton kernels.

**Current stream vs. default stream.** The **default stream** is a *specific* stream — the one work goes to when you name none ("stream 0", the null stream). The **current stream** is a per-thread *selector* for which stream new ops enqueue on; it *starts* equal to the default stream and gets reassigned by `with torch.cuda.stream(s):`. They coincide until you enter a stream context — which is exactly why a custom op must launch on the *current* stream (`getCurrentCUDAStream()`), not the default: otherwise it ignores the user's chosen stream and races. (Aside: the legacy null stream also implicitly blocks all other streams, another reason never to hard-code it; PyTorch uses per-thread default-stream semantics to avoid that global barrier.)

### 17.2 When to roll your own streams — and when it's already done for you

**TL;DR.** In a normal PyTorch/JAX model you almost never create streams — the framework already overlaps the things that matter. Add explicit streams only when you have *independent* work the framework can't tell is independent, *and* the GPU isn't already saturated by the main stream.

A stream expresses "these timelines are independent." That is only worth doing when *all three* of the following hold: the work really is independent, the GPU has spare room to run it concurrently, and the framework isn't already doing the overlap for you. In practice most of the high-value overlap is already handled:

| **Already handled — don't add streams** | **Worth your own stream(s)** |
|---|---|
| Op-by-op sequencing in a model (everything on the current stream, in order) | Input prefetch: copy the next batch H2D while computing the current one (if not already using a framework prefetcher) |
| DDP/FSDP gradient `all-reduce` / `reduce-scatter` overlapped with backward (internal comm streams) | Overlapping your *own* communication or host↔device transfers (parameter/KV-cache offload, ZeRO-Offload-style) with compute |
| cuBLAS / cuDNN internal stream use | Many *small* independent kernels that each underfill the GPU (ensembles, tiny experts, batched small ops) — genuine task parallelism |
| NCCL collectives' own stream management | Inference serving: concurrent requests, multiple replicas, multi-stream batching, CUDA MPS |
| Launch-overhead hiding via CUDA Graphs (`torch.compile`, cudagraphs) | Multi-GPU pipeline parallelism (overlap stage comm with compute) — though Megatron/DeepSpeed implement it for you |

**The decision rule.** Add an explicit stream only when all three hold: the work is genuinely independent of the main timeline; the main stream does *not* already saturate the GPU (otherwise there is no spare capacity to overlap into — the saturation point from the kernels-vs-streams discussion); and the framework isn't already overlapping it. And remember the cost side — once you add a stream you own the correctness burden (`record_stream` for cross-stream tensors, events to re-join the main stream), so only pay it when the overlap is real and *measured*. The rule of thumb: if you're writing a model, you almost certainly don't need streams; if you're writing a *data path, a serving system, or a distributed-training framework*, you probably do.

### 17.3 Beyond one GPU: pipeline parallelism

**TL;DR.** Pipeline parallelism is a *multi-GPU* algorithm — split the model's layers across GPUs and stream microbatches through the stages — not a CUDA-streams feature. But it *uses* streams underneath to overlap inter-stage communication with compute. It is the same "pipeline to hide latency" idea, now at the scale of whole GPUs.

A natural question after the streams material: is **pipeline parallelism** (PP) just streams at large scale? Not quite — it is a distributed-training algorithm that streams are only a piece of. What actually defines PP:

- **Layer partitioning across GPUs.** Each GPU ("stage") owns a contiguous group of layers.

- **A microbatch schedule.** The batch is split into microbatches that flow through the stages like an assembly line (GPipe; or 1F1B, as in PipeDream/Megatron). The schedule is framework logic, and it is what shrinks the idle *bubble* at fill/drain.
- **Inter-stage transfer via NCCL point-to-point** `send`/`recv` — activations forward, gradients backward — over NVLink within a node or InfiniBand across nodes.

None of those is a `cudaStream_t`. Where streams come in is the *plumbing*: every NCCL op runs on a stream, and frameworks put the inter-stage comm on a *separate* stream from compute so a stage can **send microbatch $m$'s activations while computing microbatch $m{+}1$** — the same `wait_stream`/event overlap as before, applied to NCCL traffic instead of an H2D copy. Two caveats: streams do *not* remove the pipeline bubble (that is the schedule's job), and the overlap they provide is *comm vs. compute*, not compute vs. compute (the compute already saturates the GPU).

**One pattern, three scales.** The cleanest way to hold all of this together: "pipeline to hide latency" shows up at every level you have met, with a different mechanism each time.

| Scale | What is pipelined | Mechanism |
|---|---|---|
| Inside one kernel | global→shared load vs. Tensor-Core math | software pipelining: `cp.async` / `num_stages` (§14) |
| One GPU, across launches | copy (or NCCL comm) vs. compute | **CUDA streams** + events |
| Across many GPUs | stage compute vs. transfer between stages | **pipeline parallelism**: microbatch schedule + NCCL P2P (run on streams) |

Table 1: CUDA streams are the single-GPU rung; pipeline parallelism is the multi-GPU rung that leans on streams internally.

## 18 Host/device memory management

**TL;DR.** `cudaMalloc` for device memory; pinned host memory for fast async copies; unified memory (`cudaMallocManaged`) for convenience at some performance cost.

Options, roughly fastest-to-easiest:

- **Explicit** — `cudaMalloc` + `cudaMemcpy`. Most control.
- **Pinned** (`cudaHostAlloc`) — non-pageable host memory; enables real `cudaMemcpyAsync` overlap and higher bandwidth.
- **Unified** (`cudaMallocManaged`) — one pointer valid on host and device; the driver migrates pages on demand. Great for prototyping and complex data structures; watch for migration overhead in hot loops (`cudaMemPrefetchAsync` helps).

## 19 Errors, debugging, and profiling

**TL;DR.** Check return codes, run `compute-sanitizer` for races/OOB, profile with Nsight Systems (timeline) and Nsight Compute (per-kernel). Same tools whether the kernel came from CUDA or Triton.

CUDA calls return `cudaError_t`; wrap them in a macro that checks and, after each launch, call `cudaGetLastError()` (launch errors are async). Key tools:

- **compute-sanitizer** — detects out-of-bounds, races, uninitialized memory (your first stop for "it gives wrong/NaN answers").
- **Nsight Systems** — whole-application timeline: where time goes, copy/compute overlap, gaps.
- **Nsight Compute (`ncu`)** — per-kernel deep dive: occupancy, memory throughput vs roofline, warp-stall reasons, bank conflicts. Use it to decide whether a kernel is memory- or compute-bound.
- **PTX/SASS inspection** (`cuobjdump`, `nvdisasm`) — read the actual instructions, just as you might dump Triton's generated PTX.

## 20 Numerical precision

**TL;DR.** FP32 is the safe default; TF32/BF16/FP16 trade precision for Tensor-Core speed; FP8 is the newest. Accumulate in FP32. Same precision choices you make with `tl.dot`'s `allow_tf32` / input dtypes.

Types you'll meet: **FP32** (default compute), **TF32** (19-bit, automatic on Tensor Cores for FP32 matmuls — big speedup, slight precision loss), **FP16**/**BF16** (16-bit; BF16 trades mantissa for FP32-range exponent, friendlier to training), and **FP8** (E4M3/E5M2, for inference and large-scale training). The standard recipe is low-precision inputs with **FP32 accumulation** to control error. `-use_fast_math` trades IEEE compliance for speed on transcendentals. These are the same tradeoffs behind Triton's dtype and `allow_tf32` flags.

## 21 Libraries — when not to write a kernel

**TL;DR.** For standard ops, call a library: cuBLAS (GEMM), cuDNN (DL ops), CUTLASS (customizable GEMM/conv), Thrust/CUB (scans, sorts, reductions). Often faster than anything hand-written — including Triton.

- **cuBLAS / cuBLASLt** — dense linear algebra; the GEMM baseline to beat.
- **cuDNN** — convolutions, attention, normalization primitives.
- **CUTLASS** — C++ templates for GEMM/conv with the tile/stage/Tensor- Core knobs exposed; use when you need a fused or custom variant.
- **CUB** (block/warp/device primitives) and **Thrust** (STL-like) — reductions, scans, sorts, selects, done right.

The discipline mirrors Triton: reach for a library first; write a custom kernel only for fusions or shapes the library doesn't serve well.

## 22 A performance methodology

**TL;DR.** Measure first. Find whether you're memory- or compute-bound (roofline), fix the dominant limiter, repeat. Don't optimize by guesswork.

A reliable loop:

1. **Profile** with `ncu` to get achieved memory bandwidth and compute throughput.
2. **Classify** via the **roofline**: compare arithmetic intensity (FLOPs per byte) to the hardware ridge point. Below it ⇒ memory-bound; above ⇒ compute-bound.
3. **If memory-bound:** improve coalescing, add shared-memory reuse, use vectorized loads, raise arithmetic intensity (fuse).
4. **If compute-bound:** use Tensor Cores / lower precision, reduce redundant work, increase ILP via register blocking.
5. **If latency-bound (low occupancy):** more resident warps, fewer registers, or pipelining.

This is the same reasoning that makes one Triton autotune config beat another — CUDA just makes you author the configs.

## 23 Triton vs. CUDA: when to use which

**TL;DR.** Stay in Triton for most fused DL kernels; drop to CUDA/CUTLASS for warp specialization, custom epilogues, exotic data movement, or the last few percent.

| Stay in Triton | Drop to CUDA / CUTLASS |
|---|---|
| Fused pointwise, softmax, layernorm, flash-attention-style kernels | Producer/consumer & warp-specialized schedules |
| Standard tiled matmul / blocked reductions | Custom epilogues, persistent kernels, exotic movement |
| Fast iteration, autotuned, portable | Hand-tuned `cp.async`/`wgmma`, last-mile perf |
| Most research and DL workloads | Library-grade kernels; features outside Triton's IR |

## 24 Where the work actually is: kernels vs. streams

**TL;DR.** Almost all GPU performance effort today goes into *kernels*, not streams — because that is where the time is spent, where the optimization ceiling is unbounded, and where new models keep creating unsolved problems. Streams are a capped, mostly-solved, framework-owned scheduling layer.

Having spent real pages on streams, it is worth saying plainly why you will spend most of your tuning effort on the *kernel* side. Effort flows to where three things coincide — the time is actually spent, the ceiling is high, and the framework has not already solved it — and all three point at kernels:

- **The time is in the kernels.** Training and inference spend the large majority of GPU time inside a few heavy kernels (matmul, attention, conv, norms, fused elementwise). Streams only reschedule *when* kernels run; they never make a kernel faster. By Amdahl's law, you optimize where the time is.
- **The stream win is capped; the kernel win is not.** Stream overlap saves at most $\text{sum} \to \max(\text{copy}, \text{compute})$ — a bounded, constant-factor gain, and only when there is exposed copy/comm to hide. Once you are compute-bound, the GPU is already busy and

there is little left to overlap. Kernel work (tiling, Tensor Cores, lower precision, fusion, in-kernel pipelining) has no such ceiling.

- **Streams are framework-owned and mostly solved.** PyTorch/JAX manage stream assignment, the stream-aware allocator, comm/compute overlap (FSDP/DDP, NCCL on side streams), and **CUDA Graphs** for launch overhead. New *models*, by contrast, keep inventing ops (new attention variants, MoE routing, quantization, scans) that have no good kernel yet — so that is where human effort is needed.
- **The hardware pushed pipelining *into* the kernel.** Tensor-Core throughput outran memory bandwidth, so feeding the math units — `cp.async`, TMA, warp specialization, shared-memory layout — became the central problem. The overlap that streams do at host scale now also has to happen *inside* the kernel.
- **Fusion removes stream opportunities.** Combining many small ops into one kernel (FlashAttention, fused optimizers, megakernels) kills global-memory round-trips and launch overhead — and eliminates the inter-kernel gaps streams might have overlapped.
- **One big kernel already saturates the GPU,** so running two large kernels concurrently via streams rarely helps. The parallelism that pays is *within* the kernel (thousands of warps), not across kernels.

**The honest caveat.** Streams still earn real attention where the GPU is *not* saturated by one kernel or where data movement is exposed: *inference serving* (concurrent small requests, MPS, multi-stream batching), *input pipelines*, and *distributed comm/compute overlap*. But even those are increasingly handled by frameworks and serving stacks — so it tends to be library authors, not most engineers, doing stream-side work. The one-line synthesis: *streams are a scheduling layer (bounded, solved, library-owned); kernels are the compute itself (unbounded, constantly renewed, and where the hardware bottleneck now lives).*

## 25 Cheat sheet: Triton ↔ CUDA

| **Triton** | **CUDA** |
|---|---|
| `tl.program_id(0)` | `blockIdx.x` |
| program instance | thread block (`blockDim` threads) |
| `tl.arange(0,BLOCK)+off` | `blockIdx.x*blockDim.x+threadIdx.x` (per thread) |
| `tl.load(p+off, mask)` | global load + `if` guard; you ensure coalescing |
| `tl.store(p+off, v, mask)` | global store + `if` guard |
| implicit tile scratch | `__shared__` + `__syncthreads()` |
| `tl.dot(a,b)` | shared-mem tiling + Tensor Cores (wmma/mma/CUTLASS) |
| `tl.sum/tl.max` | shared-mem tree reduce + `__shfl_down_sync` |
| `tl.atomic_add` | `atomicAdd` |
| `num_warps` | `blockDim/32` |
| `num_stages` | manual `cp.async` double/multi-buffering |
| `@triton.autotune` | launch-config + occupancy tuning (manual/sweep) |
| `allow_tf32`, dtypes | TF32/BF16/FP16/FP8 + FP32 accumulate |
| dump Triton PTX | `cuobjdump/nvdisasm` on your SASS |

**Where to go next.** Each worked example (§11–13) and the pipelining section (§14) is a natural

deep-dive. Given your Triton background, the highest-leverage elaboration is **tiled matmul + `cp.async` pipelining + Tensor Cores**, because it converts the most Triton intuition (`tl.dot`, `num_stages`) into explicit CUDA at once. Name a section and I'll expand it with full code and profiling.